# Studio Nx

JS 262 — 프리셋과 켜고 끄기

2026-10-18

# JavaScript 262 — 프리셋과 켜고 끄기 (`loadPresets` · `fillPresetSelect` · `applyPreset` · `toggleBypass` · `toggleSolo`)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/frontend/static/js/pages/sequencer-devices.js` |
| 작업 | S1-09 악기·이펙트 화면: 프리셋을 고르고, 바이패스와 겹 솔로를 켭니다 (FR-246 · FR-263 · FR-245) |
| 작성 | Claude (AI), 2026-10-18, 사용자 검토 |
| 이전 · 다음 | 261 상세 설정 모달 · 263 이벤트 배선과 잔손질 |

## 1. 하는 일

`loadPresets` 는 플러그인의 프리셋 목록을 `GET /plugins/{id}/presets` 로 한 번만 읽어 기억합니다. `fillPresetSelect` 는 그 목록으로 고르기 상자를 채웁니다. `applyPreset` 은 고른 프리셋 값을 초안에 올리고 저장되지 않은 상태로 표시합니다. 프리셋 자체가 서버에 저장되는 것이 아니라, 값을 초안에 올려 두었다가 Save 로 저장하는 방식입니다. `toggleBypass` 와 `toggleSolo` 는 카드의 버튼을 눌렀을 때 그 장치 값 하나를 뒤집어 저장합니다.

## 2. 언제 쓰나

| 쓰는 곳 | |
|---|---|
| `openDeviceModal` (261) | 창을 열 때 `loadPresets` 를 부릅니다 |
| `wireDialogs` (263) | 프리셋 적용 버튼과 카드 버튼을 배선합니다 |
| `wireCells` (263) | 바이패스 · 솔로 버튼 클릭에서 두 토글을 부릅니다 |

## 3. 코드

```
export async function loadPresets(plugin) {
  if (presets[plugin.plugin_id]) return presets[plugin.plugin_id];
  const list = await api.get('/plugins/' + plugin.plugin_id + '/presets');
  presets[plugin.plugin_id] = list || [];
  return presets[plugin.plugin_id];
}

/** 프리셋 고르기 <select> 채우기 */
function fillPresetSelect(list) {
  const sel = document.getElementById('device-preset');
  if (!sel) return;
  sel.innerHTML = '<option value="">— Init —</option>' + (list || []).map(function (p) {
    return '<option value="' + p.preset_id + '">' + esc(p.name) + ' (' + esc(p.source) + ')</option>';
  }).join('');
}

export function applyPreset(presetId) {
  if (!modal) return;
```

```
  const list = presets[modal.plugin.plugin_id] || [];
  const found = list.filter(function (p) { return String(p.preset_id) === String(presetId); })[0];
  if (!found) {
    modal.draft = defaultParams(modal.plugin);
    modal.device.__preset = null;
  } else {
    modal.draft = withDefaults(modal.plugin, found.params);
    modal.device.__preset = found.name;
  }
  markDirty(true);
  renderModalParams();
}

/** 이펙트 카드의 바이패스 켜기/끄기 (FR-263) */
export function toggleBypass(track, card) {
  const device = deviceOfCard(track, card);
  if (!device) return;
  const next = !device.is_bypassed;
  updateDevice(track, 'EFFECT', device, { is_bypassed: next }).then(function (saved) {
    if (saved) refreshCells(track);
  });
}

/** 악기 겹 솔로 켜기/끄기 (FR-245) */
export function toggleSolo(track, card) {
  const device = deviceOfCard(track, card);
  if (!device) return;
  const next = !device.is_solo;
  updateDevice(track, 'INSTRUMENT', device, { is_solo: next }).then(function (saved) {
    if (saved) refreshCells(track);
  });
}
```

### 문법 풀이

| 식 | 뜻 |
|---|---|
| `if (presets[plugin.plugin_id]) return ...` | 이미 읽은 목록은 다시 묻지 않습니다 |
| `p.source` | 프리셋이 어디서 왔는지(공장 · 사용자)를 보여 줍니다 |
| `String(p.preset_id) === String(presetId)` | 고른 값이 글자로 와도 맞춰 봅니다 |
| `modal.draft = withDefaults(...)` | 프리셋 값을 기본값 위에 깔아 초안으로 만듭니다 |
| `modal.device.__preset = found.name` | 카드에 보여 줄 프리셋 이름입니다 |
| `!device.is_bypassed` | 지금 상태를 뒤집습니다 |
| `.then(function (saved) { if (saved) ... })` | 저장이 성공했을 때만 다시 그립니다 |

## 4. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 프리셋을 초안에만 올림 | 값을 확인하고 Save 로 확정하는 흐름이 자연스럽습니다 |
| Init 을 첫 항목으로 | 프리셋을 고르지 않은 상태로 되돌릴 수 있습니다 |
| 목록을 기억함 | 창을 열 때마다 서버에 묻지 않습니다 |

| 선택 | 이유 |
| --- | --- |
| 바이패스 · 솔로를 따로 | 켜고 끄는 값이 서로 다르고 즉시 저장해야 소리 결과가 바로 바뀝니다 |

## 5. 시험

시험 프로젝트에서 Compressor 카드의 바이패스 버튼을 눌러 카드에 `is-bypassed` 가 붙고 `GET /tracks/4639/plugins` 의 `is_bypassed` 가 `true` 로 저장되는 것을 확인했습니다. 프리셋 목록은 시드 프리셋 0건이라 `— Init —` 한 줄만 나왔고, Init 을 고르면 값이 기본값으로 돌아오는 것을 확인했습니다.